

嵌入式系統設計與應用

Term Project

臺灣手語辨識系統之建立與樹莓派部署

鄧曉晴

一、	前言	2
二、	研究目的	2
三、	文獻回顧	3
四、	研究方法	4
五、	實驗成果	11
六、	結論	14
七、	未來展望	15
八、	文獻參考	16

嵌入式系統設計與應用

臺灣手語辨識系統之建立與樹莓派部署

01 鄧曉晴

一、前言

手語是聽障族群與外界溝通的主要方式，其中臺灣手語（Taiwan Sign Language, TSL）具有獨立語法與表達方式。然而，目前開源的手語辨識系統多以英文 ASL 為主，且常局限於靜態手勢或單一字母，缺乏針對 TSL 之動態辨識研究。本專案旨在建立一個基於 Mediapipe 姿態關鍵點與深度學習序列模型的臺灣手語動態辨識系統，並將模型壓縮為輕量化 TFLite，部署至樹莓派，使其能即時進行手語辨識並提供視覺化回饋。

二、研究目的

（一） 研究動機

目前網路上缺乏實用、開源且能即時運作的臺灣手語辨識系統。多數研究或專案僅停留於：

- 靜態手勢分類
- 英文 ASL 字母辨識
- 不支援時序手勢
- 無法在低功耗裝置上部署

因此，本研究希望：

1. 透過 Mediapipe 擷取人體姿態與手部關鍵點，建立適合時序辨識的資料。
2. 應用 LSTM/GRU 進行動態手語序列的分類。
3. 訓練可部署於樹莓派的輕量化 TFLite 模型。
4. 建立一套可即時運作的 TSL 辨識流程，提供視覺化與文字回饋

（二） 研究目的

本專案之最終目標如下：

1. 建立可辨識七種臺灣手語動作的模型。
2. 設計相對座標化特徵以提升跨角度、跨場景的泛化能力。
3. 使用 LSTM 與 GRU 比較辨識效果，選擇適合部署之模型架構。
4. 將訓練模型轉換至 TFLite，部署於樹莓派進行即時辨識。
5. 提供即時視覺化介面與中文結果顯示功能。

三、文獻回顧

（一） 手語辨識研究現況

近年手語辨識方法主要分為兩類：

- 基於影像的卷積式方法（CNN）
直接輸入影像，辨識分類。但需大量標註資料，且不易部署到低功耗裝置。
- 基於關鍵點之時序模型（LSTM/GRU）
透過姿態偵測（如 Mediapipe）取得人體、手部骨架，僅需少量資料即可辨識動態動作。

本研究採用第二類方法，其優點為：

- 抗光線、背景干擾
- 模型輕量、資料需求較低
- 適合樹莓派等低功耗設備

基於關鍵點的方法因其數據維度低和計算效率高，B. Alsharif 已證實能夠在嵌入式設備上實現實時且準確的辨識 [1]，A. S. Khalaf 等人之文獻也支持本研究採用此類方法的論點 [2]。

（二） Mediapipe 在姿態與手部追蹤之應用

Mediapipe 能提供：

- 33 個人體姿態（4 維度）
- 左右手各 21 點（3 維度）

總計：

$$33 \times 4 + 21 \times 3 \times 2 = 258 \text{ 維特徵}$$

S. Luong 採用 MediaPipe 提取關鍵點進行即時辨識 [3]，證明了此方法的優勢與高準確性；另有 Ertürk 之研究亦強調基於關鍵點的模型的輕量化特性[4]。

（三） 時序模型：LSTM 與 GRU

- LSTM

具備記憶門、輸出門、遺忘門，可處理長期依賴，但計算成本較高。

- GRU

結構較簡單，效率較高，適合在硬體受限的裝置上部署。

相關研究，例如 T.-Y. Kim 證實 GRU 在許多時序任務上可達到與 LSTM 接近甚至更高的準確率[5]，同時大幅降低了運算複雜度和訓練時間，這對於本專案追求輕量化部署至關重要。此外， Han, Y 等人於文獻中已將這類模型成功應用於基於關鍵點的手語辨識[6]，證明了其適用性。

四、研究方法

（一） 資料收集流程

使用自錄影方式製作資料集：

- 手語種類 七種：謝謝、對不起、不客氣、可以、好、幫我、幫助



圖一：手語種類示意圖

（圖片來源：<https://www.youtube.com/watch?v=nE4kuhO0l3E&t=262s>）

- 每個手語動作錄製 30 frames 為一組
- 每個動作有 30 組為訓練集、20 組包含網路手語影片，做為測試集
- 採用 Pose + Hands 分離模組確保樹莓派上可運行。

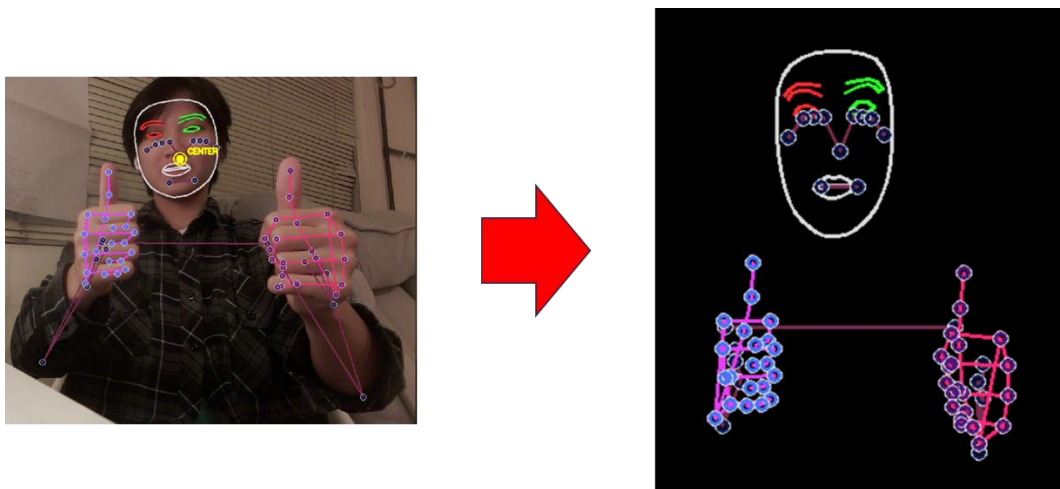
(二) 資料前處理

原始 Mediapipe 為「絕對座標 (x,y)」，根據螢幕解析度、受測者與攝影機不同距離、拍攝角度，可能導致模型無法泛化。因此本研究採用，將鼻子關鍵點座標作為中心點，所有關鍵點減去鼻子位置，存於 .npy 檔案，如下 (式 1)：

式 1：相對座標轉換

1. $\text{relative_x} = \text{point.x} - \text{nose.x}$
2. $\text{relative_y} = \text{point.y} - \text{nose.y}$
3. $\text{relative_z} = \text{point.z} - \text{nose.z}$

關鍵點擷取方式如下 (圖二)：



圖二：mediapipe 關鍵點擷取示意圖

(三) 特徵維度

Pose: $33 \times 4 = 132$

Hands: $21 \times 3 \times 2 = 126$

總計：258 維 $\times 30 \text{ frames} = [30, 258]$ 之序列輸入

（四） 模型架構

本研究使用兩種時序模型比較：

1. LSTM (Long Short-Term Memory)

LSTM 是一種改良的循環神經網路，主要目的是解決普通 RNN 在長序列學習時會遇到的「梯度消失」問題。

其核心是 記憶單元 (Cell State)，透過三個「門」進行資訊的篩選與更新：

1. 遺忘門 (Forget Gate)

判斷哪些舊資訊應該保留、哪些應該丟掉。

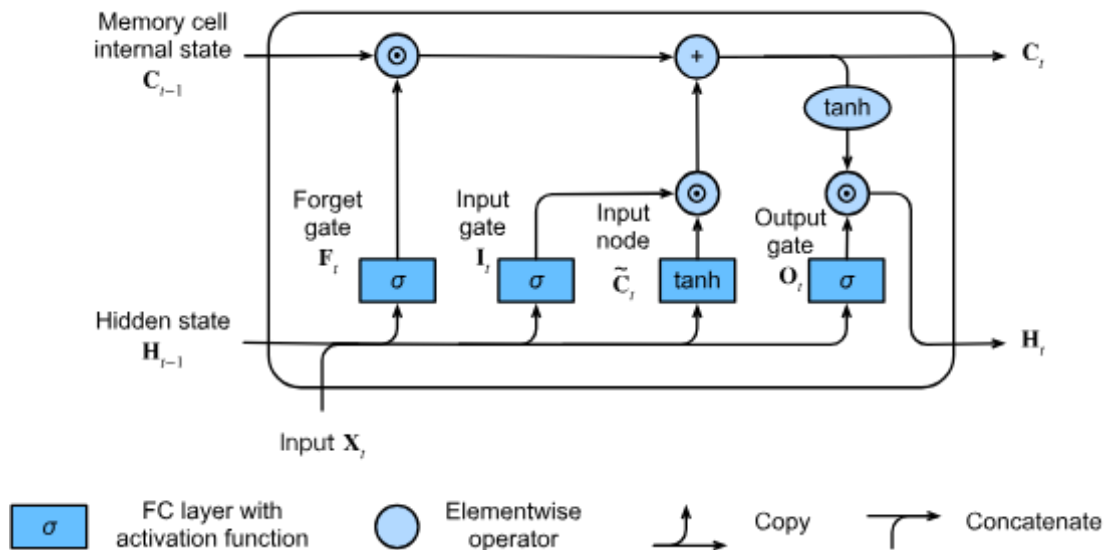
2. 輸入門 (Input Gate)

決定目前的輸入中哪些資訊可以加入記憶單元。

3. 輸出門 (Output Gate)

根據最新的記憶狀態與輸入，決定要輸出什麼。

因為具備三個門，它可以在長序列中維持穩定的記憶能力，但結構較複雜、參數量較大，因此 運算成本較高。



圖三：LSTM 模型架構

2. GRU (Gated Recurrent Unit)

GRU 是簡化版的 LSTM，將三個門合併為兩個：

1. 更新門 (Update Gate)

同時扮演 LSTM 的遺忘門 + 輸入門，控制記憶如何在每一步更新。

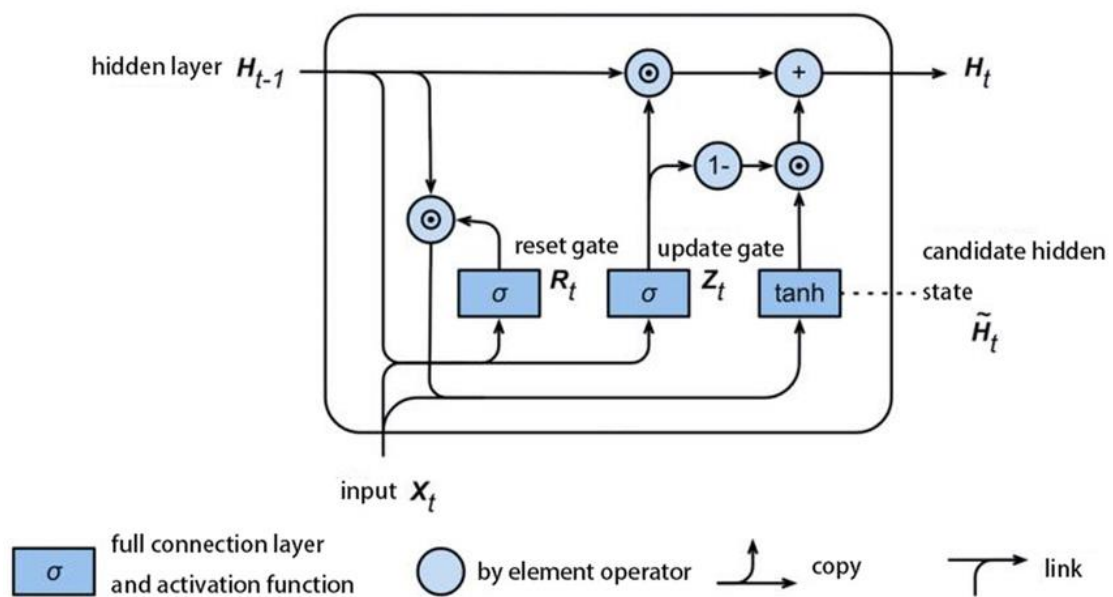
2. 重置門 (Reset Gate)

決定當前的輸入在多大程度上與先前的記憶結合。

相較於 LSTM：

- 參數更少
- 結構較簡單
- 訓練更快
- 推論速度更高

因此 GRU 很適合 硬體資源有限的環境（如樹莓派、嵌入式設備）。



圖四：GRU 模型架構圖

3. 模型訓練參數設定

本研究在訓練 GRU 手語辨識模型時採用以下超參數設定。這些參數影響模型的收斂速度、準確率以及是否容易過擬合。

- 基本訓練參數

Epochs : 2000, Batch Size : 32, Validation Split : 0.1

- 優化器與學習率

Optimizer : Adam, Learning Rate : 0.0005

- 模型正規化策略

Dropout : 0.2

- Callback 設定

- EarlyStopping

驗證損失 (val_loss)、patience = 200

- ModelCheckpoint

儲存驗證準確度 (val_categorical_accuracy) 最佳的模型。

- TensorBoard

用於視覺化訓練曲線。

(五) 訓練流程

1. 資料準備與預處理：載入 .npy 格式的手語關鍵點數據，並進行正規化與形狀調整，使模型輸入維度一致。
2. 模型訓練：訓練 LSTM 與 GRU 模型
3. 模型評估：輸出混淆矩陣及準確率，驗證模型的識別能力。
4. 模型輕量化與部署：將 GRU 模型轉換為 TFLite，部署於樹莓派
5. 系統驗證：測試即時手語辨識，確認模型能正確分辨相似動作並保持高精度。

(六) 模型架構程式碼

- LSTM

```
139 # ===== LSTM 架構 =====
140 def build_model(input_shape):
141     """建立 LSTM 神經網路模型"""
142     print("\n建立模型...")
143
144     model = Sequential()
145     model.add(LSTM(64, return_sequences=True, activation='relu', unroll=True, input_shape=input_shape))
146     model.add(Dropout(0.2))
147     model.add(LSTM(128, return_sequences=True, activation='relu', unroll=True))
148     model.add(Dropout(0.2))
149     model.add(LSTM(64, return_sequences=False, activation='relu', unroll=True))
150     model.add(Dropout(0.2))
151     model.add(Dense(64, activation='relu'))
152     model.add(Dropout(0.2))
153     model.add(Dense(32, activation='relu'))
154     model.add(Dense(actions.shape[0], activation='softmax'))
155
156     optimizer = Adam(learning_rate=0.0005)
157     model.compile(optimizer=optimizer,
158                   loss='categorical_crossentropy',
159                   metrics=['categorical_accuracy'])
160
161     return model
```

1. 三層 LSTM：

- 第一層 64 units，return_sequences=True。
- 第二層 128 units，深度學習序列特徵。
- 第三層 64 units，return_sequences=False，將時序壓縮成向量。

2. Dropout：

每層 0.2，防止過擬合。

3. Dense 層：

- 64 → 32 units，非線性映射。
- 最後 softmax 輸出分類。

- GRU

```
110 # ===== GRU 架構 =====
111 def build_model(input_shape):
112     """建立 GRU 神經網路模型"""
113     print("\n建立模型...")
114
115     model = Sequential()
116     model.add(GRU(64, return_sequences=True, activation='relu', unroll=True, input_shape=input_shape))
117     model.add(Dropout(0.2))
118     model.add(GRU(128, return_sequences=True, activation='relu', unroll=True))
119     model.add(Dropout(0.2))
120     model.add(GRU(64, return_sequences=False, activation='relu', unroll=True))
121     model.add(Dropout(0.2))
122     model.add(Dense(64, activation='relu'))
123     model.add(Dropout(0.2))
124     model.add(Dense(32, activation='relu'))
125     model.add(Dense(actions.shape[0], activation='softmax'))
126
127     from tensorflow.keras.optimizers import Adam
128
129     optimizer = Adam(learning_rate=0.0005) # 可調 1e-2 ~ 1e-5
130     model.compile(optimizer=optimizer,
131                   loss='categorical_crossentropy',
132                   metrics=['categorical_accuracy'])
133
134     return model
```

1. 三層 GRU：

- 第一層 64 units，return_sequences=True 保留序列資訊，方便後續層處理。
- 第二層 128 units，進一步抽取時序特徵。
- 第三層 64 units，return_sequences=False，將序列壓縮成單一向量輸入 Dense 層。

2. Dropout：每層 GRU 後加入 0.2 的 Dropout，減少過擬合。

3. Dense 層：

- 64 → 32 units，用於非線性特徵轉換。
- 最後輸出層為 softmax，對应手語分類數量。

五、實驗成果

(一) LSTM 與 GRU 在不同座標系統下之比較

本次實驗針對相同的資料前處理與輸入特徵 (258 維)，分別對 LSTM 與 GRU 進行訓練。比較結果如下 (表一)：

表一：不同模型與資料格式的驗證準確率比較：

Model \ Validation acc	絕對座標	相對座標
LSTM	0.17	0.70
GRU	0.17	0.82

由 (表一) 可見：

1. 絕對座標資料表現極差

不論是 LSTM 或 GRU，其驗證準確率 (val_categorical_accuracy) 皆接近隨機猜測 ($0.17 \approx \frac{1}{7}$)，顯示模型無法從不同受試者的絕對空間位置中學習出穩定的時序關聯性，故將絕對座標資料從實驗中排除。

2. 相對座標資料有效改善模型辨識能力

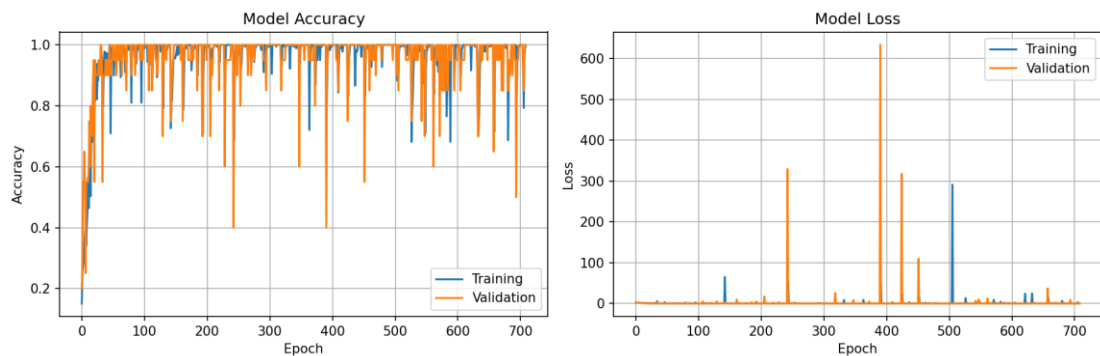
為了消除人體大小、距離攝影機遠近等因素的干擾，本研究將手語關鍵點資料轉換為相對座標。此方法有效讓模型更專注於手勢本身的動作模式，提升辨識能力。實驗結果如下：

- LSTM 測試集準確度從 17% 提升至 71%
- GRU 測試集準確度從 17% 提升至 82%

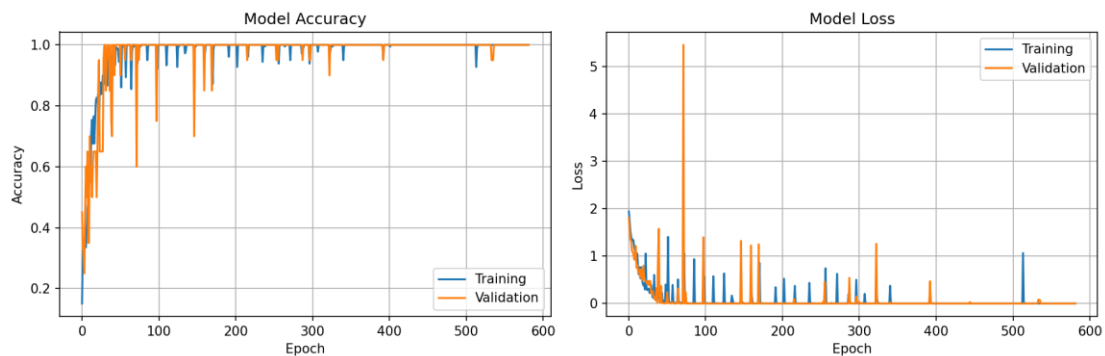
可見 GRU 準確率勝於 LSTM。

3. 模型比較與觀察

LSTM 之 learning curve 如下（圖五），對比於 GRU（圖六）：



圖五：相對座標下 LSTM training curve



圖六：相對座標下 GRU training curve

從曲線可以觀察到：

- GRU 模型收斂速度較穩定，noise 較小。
- LSTM 曲線震盪較明顯，學習過程不如 GRU 平滑。

由此可驗證 GRU 結構較簡單、訓練更穩定，對於本研究的中小型資料集，更適合用於訓練。

（二） 使用 Adam 後最終結果

在確定 GRU 搭配相對座標較佳後，本研究進一步調整訓練設定：

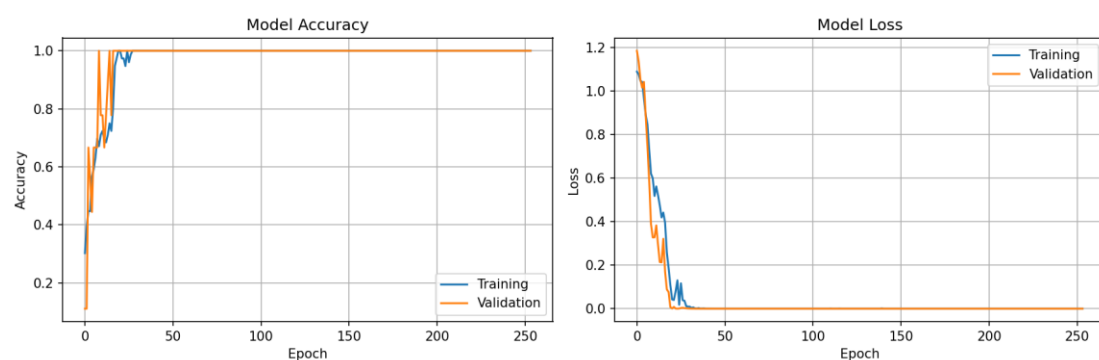
- Optimizer：Adam
- Learning Rate：0.0005
- Early Stopping 與 Dropout 0.2 防止過擬合

經調整後，模型的最終測試準確率提升至 **97.86%**，測試集報告如下（圖七）：

整體準確率: 97.86%				
分類報告:				
	precision	recall	f1-score	support
謝謝	0.87	1.00	0.93	20
幫我	1.00	0.85	0.92	20
對不起	1.00	1.00	1.00	20
可以	1.00	1.00	1.00	20
好	1.00	1.00	1.00	20
不客氣	1.00	1.00	1.00	20
幫助	1.00	1.00	1.00	20
accuracy			0.98	140
macro avg	0.98	0.98	0.98	140
weighted avg	0.98	0.98	0.98	140

圖七：測試集報告

優化後的 training curve 如下（圖八）：



圖八：使用 Adam 優化器下 GRU 之 training curve

由（圖八）可見：

- 收斂速度明顯加快
從 600 epoch 下降至 100 epoch 以內即收斂完畢
- Training curve 更穩定
- 訓練過程無過擬合跡象

training 與 validation 曲線趨勢一致，說明在 Adam 與適當的學習率設定下，GRU 能有效擷取手勢的時序特徵並迅速收斂至高準確度。

（三） 樹梅派手語辨識系統成果展示

本研究將訓練完成的 GRU 模型轉換為輕量化 TFLite 模型，成功部署於樹莓派系統中。在實際應用，系統能即時辨識手語動作，並將結果對應至畫面顯示如（圖九），即使是僅有左右手前後差異的相近手勢，也能被模型正確分辨，顯示出 GRU 模型在中小型資料集上的高精確度與穩定性。



圖九：手語辨識系統成果

六、結論

本研究成功建立一套針對臺灣手語之即時辨識系統，成果如下：

1. 建立基於 258 維關鍵點、30 frames 的時序辨識架構
2. 成功採用相對座標系統，大幅改善泛化能力
3. 比較 LSTM、GRU，最終選擇 GRU 適合部署
4. 模型壓縮為 TFLite 並成功部署於樹莓派
5. 最終辨識準確率達 **97.86%**，可於樹梅派即時輸出中文回饋

本研究證實：

Mediapipe + 相對座標 + GRU 是一條有效率且高效能的手語辨識途徑。

七、未來展望

未來可延伸方向如下：

1. 擴增資料集

目前僅七種手語，可擴展至日常生活中更完整的手語詞彙。

2. 串接語言模型（LLM）

將手語序列轉換為完整句子，提升溝通便利性。

3. 雙手複雜語法辨識

TSL 存在許多雙手、空間語法，需更強大的時序模型。

4. 多攝影機與 3D 辨識

可改善遮蔽問題，提升自然性。

5. 導入量化與多執行緒提升樹莓派效能

使 FPS 更順暢，接近即時對話能力。但 FPS 主要受限於樹莓派本身硬體限制，若要進一步發展即時系統，可能考慮直接更換硬體配置，是更為效率且實際的作法。

八、文献参考

- [1] B. Alsharif *et al.*, “Real-Time American Sign Language Interpretation Using Deep Learning and Keypoint Tracking,” *Sensors*, vol. 25, no. 7, p. 2138, Mar. 2025, doi: 10.3390/s25072138.
- [2] S. Khalaf *et al.*, “A Comparative Study of Hand Gesture Recognition Devices in the Context of Game Design,” in *Proc. 2019 ACM Int. Conf. on Interactive Surfaces and Spaces (ISS '19)*, Utrecht, Netherlands, Nov. 2019, pp. 397–402, doi: 10.1145/3343055.3360758.
- [3] S. Luong, “Video Sign Language Recognition using Pose Extraction and Deep Learning Models,” Master's Projects, San Jose State University, San Jose, CA, USA, 2023. doi: 10.31979/etd.jm4c-myd4.
- [4] Ertürk, K., Altınışık, F., Sarıaltın, İ., & Gerek, Ö. N. (2025). *TSLFormer: A lightweight transformer model for Turkish sign language recognition using skeletal landmarks*. arXiv. <https://doi.org/10.48550/arXiv.2505.07890>
- [5] T.-Y. Kim *et al.*, “Comparative Analysis and Validation of LSTM and GRU Models for Predicting Annual Mean Sea Level in the East Sea: A Case Study of Ulleungdo Island,” *Appl. Sci.*, vol. 15, no. 20, p. 11067, Oct. 2025, doi: 10.3390/app152011067.
- [6] Han, Y., Han, Y., & Jiang, Q. (2025). A study on the STGCN-LSTM sign language recognition model based on phonological features of sign language. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2025.3560779>